Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

Отчет о проделанной работе.
Кривые Безье, B-сплайны и P-сплайны

по дисциплине «Вычислительная математика»

Выполнил
студент гр. 5130903/40001 М.М. Алексеев

Руководитель
доцент, д.т.н. Е.Г. Хитров

«_____» _____ 2026 г.

Санкт-Петербург
2026

Содержание

1 Введение . . . 3
2 Кривые Безье . . . 4
2.1 Определение и общая формула . . . 4
2.2 Квадратичная кривая Безье . . . 4
2.3 Кубическая кривая Безье . . . 5
2.4 Склеивание кривых Безье и условия гладкости . . . 6
3 B-сплайны . . . 8
3.1 Определение . . . 8
3.2 Расслабленный равномерный кубический B-сплайн . . . 8
3.3 Интерполяция B-сплайнами . . . 9
3.4 Программная реализация . . . 9
4 P-сплайны . . . 12
4.1 Определение и мотивация . . . 12
4.2 Функционал качества . . . 12
4.3 Матричная форма и решение . . . 13
4.4 Роль параметра λ . . . 13
4.5 Связь с классическим сглаживающим сплайном . . . 13
4.6 Программная реализация . . . 14
5 Заключение . . . 16
6 Список источников . . . 17

1 Введение

Задача построения гладких кривых и аппроксимации функций по дискретным данным является одной из центральных в вычислительной математике. Классический аппарат кубических интерполяционных сплайнов, обеспечивающий непрерывность второй производной на всей области определения, широко применяется на практике, однако обладает рядом существенных ограничений.

Проблема 1: отсутствие локального контроля. При изменении одной точки данных интерполирующий кубический сплайн перестраивается глобально. Для задач компьютерного проектирования (CAD), компьютерной графики, проектирования шрифтов и анимации это неприемлемо: необходима возможность локальной коррекции формы кривой без влияния на отдалённые участки.

Проблема 2: интерполяция не всегда является целью. В ряде задач требуется не проведение кривой через заданные точки, а управление её формой посредством контрольных точек, играющих роль «притягивающих» элементов.

Проблема 3: переобучение при наличии шума. Если данные зашумлены, интерполяционный сплайн воспроизводит шум, что приводит к нежелательным осцилляциям. Необходим механизм штрафа за негладкость, позволяющий находить баланс между точностью приближения и гладкостью результата.

Указанные проблемы последовательно решаются тремя конструкциями: *кривые Безье* (элементарные параметрические кубические сегменты), *B-сплайны* (составные кривые с локальным контролем и автоматической C^2-гладкостью) и *P-сплайны* (B-сплайновая аппроксимация со штрафом на конечные разности коэффициентов).

Целью настоящей работы является изложение теоретических основ перечисленных методов, а также их программная реализация на языке Python с использованием библиотеки NumPy.

2 Кривые Безье

2.1 Определение и общая формула

Кривая Безье — параметрическая кривая, задаваемая набором дискретных контрольных точек, которые определяют плавную непрерывную кривую [1]. Первая и последняя контрольные точки являются концами кривой; остальные точки, как правило, на кривой не лежат.

Кривая Безье первого порядка (линейная интерполяция между точками P_0 и P_1) определяется формулой:

$$B(t) = P_0 + t(P_1 - P_0) = (1-t)P_0 + tP_1, \quad 0 \le t \le 1. \tag{1}$$

Кривая Безье n-го порядка задаётся в общем виде через полиномы Бернштейна:

$$B(t) = \sum_{i=0}^{n} C_n^i (1-t)^{n-i} t^i P_i, \tag{2}$$

где $C_n^i = \frac{n!}{i!(n-i)!}$ — биномиальные коэффициенты, P_i — контрольные точки.

2.2 Квадратичная кривая Безье

Кривая Безье второго порядка определяется тремя контрольными точками P_0, P_1, P_2. Она может быть интерпретирована как линейная интерполяция между двумя кривыми Безье первого порядка: от P_0 к P_1 и от P_1 к P_2:

$$B(t) = (1-t)[(1-t)P_0 + tP_1] + t[(1-t)P_1 + tP_2]. \tag{3}$$

Раскрывая скобки, получаем:

$$B(t) = (1-t)^2 P_0 + 2(1-t)tP_1 + t^2 P_2, \quad 0 \le t \le 1. \tag{4}$$

Первая и вторая производные квадратичной кривой Безье:

$$B'(t) = 2(1-t)(P_1 - P_0) + 2t(P_2 - P_1), \tag{5}$$

$$B''(t) = 2(P_2 - 2P_1 + P_0). \tag{6}$$

Геометрическое построение квадратичной кривой Безье показано на рис. Рисунок 1. Точка Q_0 линейно перемещается от P_0 к P_1, точка Q_1 — от P_1 к P_2, а точка B на кривой является линейной интерполяцией между Q_0 и Q_1 при заданном параметре t.

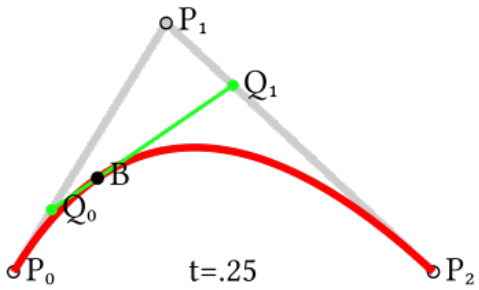

Рисунок 1 — Построение квадратичной кривой Безье при $t = 0.25$

2.3 Кубическая кривая Безье

Кривая Безье третьего порядка задаётся четырьмя контрольными точками P_0, P_1, P_2, P_3:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t)t^2 P_2 + t^3 P_3, \quad 0 \le t \le 1. \quad (7)$$

Кривая начинается в точке P_0, двигаясь в направлении P_1, и приходит к P_3 со стороны P_2.

Первая производная:

$$B'(t) = 3(1-t)^2(P_1 - P_0) + 6(1-t)t(P_2 - P_1) + 3t^2(P_3 - P_2). \quad (8)$$

Значения на концах: $B'(0) = 3(P_1 - P_0)$, $B'(1) = 3(P_3 - P_2)$.

Вторая производная:

$$B''(t) = 6(1-t)(P_2 - 2P_1 + P_0) + 6t(P_3 - 2P_2 + P_1). \quad (9)$$

Значения на концах: $B''(0) = 6(P_2 - 2P_1 + P_0)$, $B''(1) = 6(P_3 - 2P_2 + P_1)$.

Геометрическое построение кубической кривой Безье алгоритмом де Кастельо показано на рис. Рисунок 2.

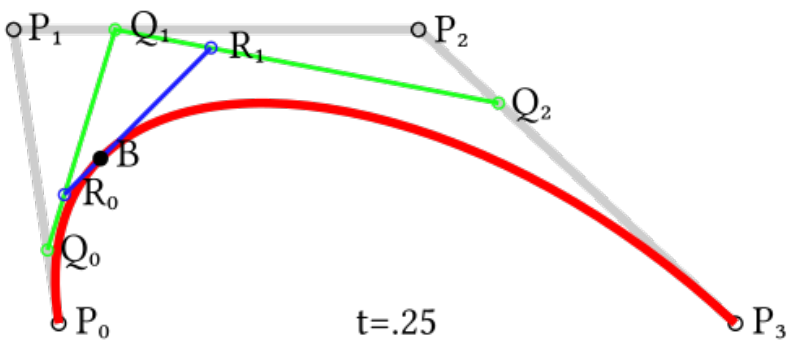

Рисунок 2 — Построение кубической кривой Безье при $t = 0.25$

2.4 Склеивание кривых Безье и условия гладкости

Для построения сложных кривых необходимо склеивать несколько сегментов Безье. Пусть первая кривая задана контрольными точками P_0, P_1, P_2, P_3, а вторая — Q_0, Q_1, Q_2, Q_3, причём $P_3 = Q_0 = S$ (точка стыка).

Непрерывность C^0 требует лишь $P_3 = Q_0$. Кривая непрерывна, но в точке стыка возможен угол.

Непрерывность C^1 (совпадение первых производных) достигается при условии $S - P_2 = Q_1 - S$, т.е.
S является серединой отрезка $\overline{P_2Q_1}$.

Непрерывность C^2 (совпадение первых и вторых производных) требует дополнительно:

$$P_1 - 2P_2 + S = S - 2Q_1 + Q_2. \tag{10}$$

Геометрически это условие эквивалентно формированию так называемой *A-рамки* (A-frame), показанной на рис. Рисунок 3. A-рамка — это конфигурация, в которой S является серединой $\overline{P_2Q_1}$, P_2 — серединой $\overline{P_1A}$, а Q_1 — серединой $\overline{Q_2A}$, где A — вершина рамки [1].

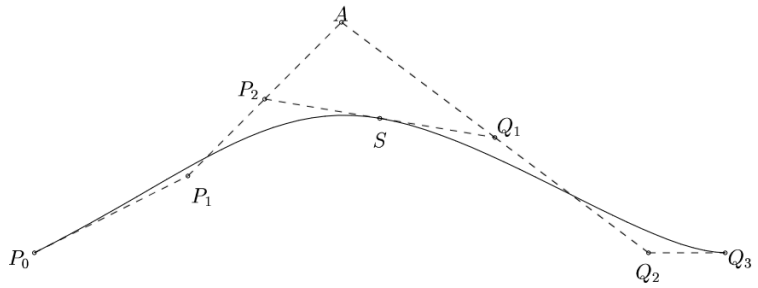

Рисунок 3 — А-рамка: геометрический критерий C^2-гладкости склейки двух кривых Безье

Совпадение третьих производных привело бы к тому, что оба сегмента являлись бы частями одного кубического полинома, что делает кусочную конструкцию бессмысленной.

3 B-сплайны

3.1 Определение

B-сплайн (basis spline) — тип сплайновой функции, которая может быть однозначно выражена в виде разложения по базису [4]. Концепция восходит к идеям Н.И. Лобачевского, а формализована И. Шёнбергом. B-сплайны широко используются в CAD- и CAM-системах, компьютерной графике и статистическом сглаживании.

Базисная B-сплайновая функция $B_i^p(t)$ степени p определяется рекурсивным соотношением Кокса — де Бора [4]:

$$B_i^0(t) = \begin{cases} 1, t_i \leq t < t_{i+1} \\ 0, \text{иначе} \end{cases} \tag{11}$$

$$B_i^p(t) = \frac{t - t_i}{t_{i+p} - t_i} B_i^{p-1}(t) + \frac{t_{i+p+1} - t}{t_{i+p+1} - t_{i+1}} B_{i+1}^{p-1}(t), \tag{12}$$

где $t_0, t_1, ...$ — вектор узлов (knot vector). Первое слагаемое изменяется от 0 до 1 при t от t_i до t_{i+p}, второе — от 1 до 0 при t от t_{i+1} до t_{i+p+1}.

Каждый B-сплайн степени p отличен от нуля лишь на $p + 2$ последовательных узловых интервалах, что обеспечивает *компактный носитель* и, как следствие, локальность влияния.

Произвольная сплайн-функция степени p на заданной сетке узлов представляется в виде линейной комбинации:

$$S(x) = \sum_i c_i B_i^p(x). \tag{13}$$

3.2 Расслабленный равномерный кубический B-сплайн

Для построения кривой с контрольными точками $B_0, B_1, ..., B_n$ и автоматической C^2-гладкостью используется следующий алгоритм [1]:

1. Каждая сторона контрольного многоугольника делится на три равные части, образуя точки деления $\frac{2}{3}B_{i-1} + \frac{1}{3}B_i$ и $\frac{1}{3}B_{i-1} + \frac{2}{3}B_i$.

2. При каждой внутренней вершине B_i ближайшие точки деления соединяются отрезком, середина которого определяет точку склейки:

$$S_i = \frac{1}{6}B_{i-1} + \frac{2}{3}B_i + \frac{1}{6}B_{i+1}, \quad i = 1, ..., n-1, \tag{14}$$

с граничными условиями $S_0 = B_0$, $S_n = B_n$ (расслабленные концы, $B''(0) = B''(n) = 0$).

3. Для i-го сегмента (от S_{i-1} к S_i) четыре контрольные точки Безье:

$$S_{i-1}, \quad \frac{2}{3}B_{i-1} + \frac{1}{3}B_i, \quad \frac{1}{3}B_{i-1} + \frac{2}{3}B_i, \quad S_i. \tag{15}$$

При каждой вершине B_i точки деления и S_i автоматически образуют А-рамку, что гарантирует C^2-гладкость в каждой точке склейки.

3.3 Интерполяция B-сплайнами

Если заданы точки данных $S_i = f(x_i)$, $i = 0, ..., n$, через которые кривая должна проходить, контрольные точки B_i определяются из трёхдиагональной системы:

$$B_{i-1} + 4B_i + B_{i+1} = 6S_i, \quad i = 1, ..., n-1, \tag{16}$$

при $B_0 = S_0$, $B_n = S_n$. Матрица системы является строго диагонально доминантной ($4 > 1 + 1$), что гарантирует единственность решения. Система решается методом прогонки (алгоритм Томаса) за $O(n)$ операций.

3.4 Программная реализация

Ниже приведена реализация метода прогонки и интерполирующего B-сплайна на языке Python.

```python
def solve_tridiag(sub, main, sup, rhs):
    """
    Метод прогонки для трёхдиагональной системы.
    sub  : нижняя диагональ  (длина m-1)
    main : главная диагональ (длина m)
    sup  : верхняя диагональ (длина m-1)
    rhs  : правая часть       (длина m)
    """
    m = len(main)
    c = sup.astype(float).copy()
    d = rhs.astype(float).copy()
    a = sub.astype(float).copy()
    b = main.astype(float).copy()
```

```
    # Прямой ход
    for i in range(1, m):
        w = a[i - 1] / b[i - 1]
        b[i] -= w * c[i - 1]
        d[i] -= w * d[i - 1]

    # Обратный ход
    x = np.zeros(m)
    x[-1] = d[-1] / b[-1]
    for i in range(m - 2, -1, -1):
        x[i] = (d[i] - c[i] * x[i + 1]) / b[i]
    return x
```

Листинг 1. Метод прогонки (алгоритм Томаса).

```
def bspline_interpolation(x_knots, y_knots):
    n = len(x_knots) - 1
    S = y_knots.copy()

    # Решаем B_{i-1} + 4·B_i + B_{i+1} = 6·S_i
    m = n - 1
    rhs = 6.0 * S[1:n].copy()
    rhs[0]  -= S[0]
    rhs[-1] -= S[n]

    B_inner = solve_tridiag(
        sub=np.ones(m - 1),
        main=4.0 * np.ones(m),
        sup=np.ones(m - 1),
        rhs=rhs
    )
    B = np.empty(n + 1)
    B[0], B[1:n], B[n] = S[0], B_inner, S[n]

    # Точки склейки
    S_joints = np.empty(n + 1)
    S_joints[0], S_joints[n] = B[0], B[n]
    for i in range(1, n):
        S_joints[i] = (B[i-1] + 4*B[i] + B[i+1]) / 6.0

    # Контрольные точки Безье для каждого сегмента
    segments = []
    for i in range(1, n + 1):
        P0 = S_joints[i - 1]
        P1 = (2.0 * B[i - 1] + B[i]) / 3.0
        P2 = (B[i - 1] + 2.0 * B[i]) / 3.0
        P3 = S_joints[i]
        segments.append((P0, P1, P2, P3))

    def evaluate(x):
        x = np.asarray(x, dtype=float)
        y = np.zeros_like(x)
        for i, (P0, P1, P2, P3) in enumerate(segments):
            mask = (x >= x_knots[i]) & (x <= x_knots[i + 1])
            t = (x[mask] - x_knots[i]) / (x_knots[i+1] - x_knots[i])
            y[mask] = ((1-t)**3*P0 + 3*(1-t)**2*t*P1
                       + 3*(1-t)*t**2*P2 + t**3*P3)
        return y
```

```
    return evaluate
```

Листинг 2. Интерполяция расслабленным кубическим B-сплайном.

4 P-сплайны

4.1 Определение и мотивация

P-сплайн (penalized B-spline) — вид B-сплайна, в котором коэффициенты определяются частично данными аппроксимации и частично дополнительной штрафной функцией, призванной обеспечить гладкость и предотвратить переобучение [2]. P-сплайн строится на основе расслабленного кубического B-сплайна (relaxed cubic spline), узлы которого имеют постоянное расстояние между друг другом.

Ключевая идея P-сплайнов состоит в том, чтобы использовать заведомо избыточное число равноотстоящих базисных B-сплайнов и компенсировать возникающую чрезмерную гибкость штрафом на разности коэффициентов.

Пусть $B_1(x), B_2(x), ..., B_K(x)$ — базисные B-сплайны степени d с равноотстоящими узлами. Аппроксимация сплайновой функцией имеет вид:

$$f(x) = \sum_{j=1}^{K} \alpha_j B_j(x). \tag{17}$$

4.2 Функционал качества

Минимизируемый функционал состоит из невязки и штрафа:

$$Q(\boldsymbol{\alpha}) = \underbrace{\sum_{i=1}^{n} (y_i - f(x_i))^2}_{\text{невязка}} + \lambda \underbrace{\sum_{j=q+1}^{K} \left(\Delta^q \alpha_j\right)^2}_{\text{штраф}}, \tag{18}$$

где $\lambda > 0$ — параметр сглаживания, Δ^q — оператор конечных разностей порядка q.

Наиболее популярен штраф второго порядка ($q = 2$):

$$\Delta^2 \alpha_j = \alpha_j - 2\alpha_{j-1} + \alpha_{j-2}. \tag{19}$$

4.3 Матричная форма и решение

Обозначим $\boldsymbol{B}$ — матрицу $n \times K$ значений базисных функций $[\boldsymbol{B}]_{ij} = B_j(x_i)$, а $\boldsymbol{D}_q$ — матрицу конечных разностей порядка q размера $(K-q) \times K$. Тогда функционал Уравнение 18 принимает матричный вид:

$$Q(\boldsymbol{\alpha}) = (\boldsymbol{y} - \boldsymbol{B}\boldsymbol{\alpha})^T(\boldsymbol{y} - \boldsymbol{B}\boldsymbol{\alpha}) + \lambda \boldsymbol{\alpha}^T \boldsymbol{D}_q^T \boldsymbol{D}_q \boldsymbol{\alpha}. \tag{20}$$

Приравнивая градиент к нулю, получаем решение — обобщённый метод наименьших квадратов:

$$\hat{\boldsymbol{\alpha}} = \left(\boldsymbol{B}^T\boldsymbol{B} + \lambda \boldsymbol{D}_q^T \boldsymbol{D}_q\right)^{-1} \boldsymbol{B}^T \boldsymbol{y}. \tag{21}$$

4.4 Роль параметра λ

При $\lambda \to 0$ штраф исчезает и P-сплайн вырождается в обычную B-сплайновую аппроксимацию без регуляризации, что может привести к переобучению. При $\lambda \to \infty$ штраф доминирует: для разностей второго порядка все $\Delta^2 \alpha_j \to 0$, т.е.
коэффициенты образуют арифметическую прогрессию, и сплайн стремится к прямой линии.

Оптимальное значение λ определяется автоматически с помощью обобщённой кросс-валидации (GCV), критерия Акаике (AIC) или байесовских методов.

4.5 Связь с классическим сглаживающим сплайном

Классический сглаживающий сплайн минимизирует функционал с интегральным штрафом:

$$\sum_{i=1}^{n} (y_i - f(x_i))^2 + \lambda \int_a^b (f''(x))^2 dx. \tag{22}$$

При достаточно большом числе равноотстоящих узлов дискретный штраф $\sum \left(\Delta^2 \alpha_j\right)^2$ аппроксимирует интеграл $\int (f'')^2 dx$ [3]. Однако дискретная форма обладает существенным преимуществом: степень B-сплайнов и порядок штрафа могут комбинироваться независимо, что невозможно в классическом подходе.

4.6 Программная реализация

```
def bspline_basis(x, knots, degree=3):
    x = np.asarray(x, dtype=float)
    knots = np.asarray(knots, dtype=float)
    N = len(x)

    # Степень 0: кусочно-постоянные функции
    B = np.zeros((N, len(knots) - 1))
    for i in range(len(knots) - 1):
        if knots[i] < knots[i + 1]:
            B[(x >= knots[i]) & (x < knots[i+1]), i] = 1.0
    # Обработка правой границы
    for i in range(len(knots) - 2, -1, -1):
        if knots[i] < knots[i + 1]:
            B[x == knots[-1], i] = 1.0
            break

    # Рекурсия по степеням 1..degree
    for d in range(1, degree + 1):
        B_new = np.zeros((N, len(knots) - 1 - d))
        for i in range(len(knots) - 1 - d):
            denom1 = knots[i+d] - knots[i]
            denom2 = knots[i+d+1] - knots[i+1]
            if denom1 > 0:
                B_new[:, i] += (x - knots[i]) / denom1 * B[:, i]
            if denom2 > 0:
                B_new[:, i] += (knots[i+d+1] - x) / denom2 * B[:, i+1]
        B = B_new
    return B
```

Листинг 3. Вычисление матрицы B-сплайнового базиса по рекурсии Кокса — де Бора.

```
def diff_matrix(K, order=2):
    """Матрица конечных разностей порядка order."""
    D = np.eye(K)
    for _ in range(order):
        D = D[1:, :] - D[:-1, :]
    return D


def pspline_fit(x_data, y_data, a, b,
                n_basis_internal=25, degree=3,
                penalty_order=2, lam=1.0):
    knots = make_knot_vector(a, b, n_basis_internal, degree)
    K = len(knots) - degree - 1

    Bmat = bspline_basis(x_data, knots, degree)
    D = diff_matrix(K, penalty_order)

    # (B^T B + λ D^T D) α = B^T y
    A = Bmat.T @ Bmat + lam * D.T @ D
    alpha = np.linalg.solve(A, Bmat.T @ y_data)

    def evaluate(x):
        return bspline_basis(x, knots, degree) @ alpha
```

```
    return evaluate, alpha
```

Листинг 4. Подгонка P-сплайна со штрафом на конечные разности.

5 Заключение

В настоящей работе рассмотрены три уровня аппарата сплайновых кривых, образующих логическую цепочку: кусочный параметрический сегмент (кривая Безье) — составная кривая с автоматической C^2-гладкостью и локальным контролем (B-сплайн) — статистическое сглаживание с регуляризацией (P-сплайн).

Кривые Безье обеспечивают удобное параметрическое описание отдельного кубического сегмента через контрольные точки, однако лишены локального контроля и не гарантируют гладкость при склеивании.

B-сплайны решают обе указанные проблемы: структура A-рамки автоматически обеспечивает C^2-непрерывность в точках склейки, а компактный носитель базисных функций (рекурсия Кокса — де Бора) гарантирует, что сдвиг одной контрольной точки влияет не более чем на четыре соседних сегмента.

P-сплайны, предложенные Эйлерсом и Марксом (1996), дополняют B-сплайновый базис штрафом на конечные разности коэффициентов, что позволяет использовать заведомо избыточное число равноотстоящих узлов и управлять балансом «точность—гладкость» единственным параметром λ. Выбор оптимального λ осуществляется автоматически с помощью обобщённой кросс-валидации (GCV).

Все описанные методы реализованы на языке Python с использованием только библиотеки NumPy, без привлечения сторонних сплайновых модулей.

6 Список источников

1. de Boor C. A Practical Guide to Splines. — Revised Edition. — New York: Springer, 2001. — 346 p.
2. Eilers P.H.C., Marx B.D. Flexible smoothing with B-splines and penalties // Statistical Science. — 1996. — Vol. 11, No. 2. — P. 89–121.
3. Eilers P.H.C., Marx B.D. Splines, knots, and penalties // Wiley Interdisciplinary Reviews: Computational Statistics. — 2010. — Vol. 2, No. 6. — P. 637–653.
4. B-spline // Wikipedia. — URL: https://en.wikipedia.org/wiki/B-spline (дата обращения: 08.04.2026).
5. Bézier curve // Wikipedia. — URL: https://en.wikipedia.org/wiki/Bézier_curve (дата обращения: 08.04.2026).